

Rules, Custom Entities & Chatbots

Post-Assessment 4 · 8 questions · 31 points

Name: _____

Date: _____

Score: / 31

This is a **post-assessment**. It shows what you learned in this unit. Try every question, and check your answers if you have time.

Work on your own, without a computer. You have about **40 minutes**. When a question asks what a program prints, write exactly what Python would print, one line of output per answer line. If the code would crash, write the name of the error (for example, `TypeError`). Show your thinking; partial credit is given.

In every spaCy question, assume this code has already run: `import spacy` and `nlp = spacy.load("en_core_web_sm")`

1. Does it match?

4 points

```
pattern = [{"LOWER": "good"}, {"LOWER": "morning"}]
```

For each text, write **Yes** if the pattern finds a match, or **No** if it doesn't.

Text	Match?
"Good morning"	
"goodmorning"	
"good Morning!"	
"good night"	

2. Write the pattern

3 points

Write a Matcher pattern (a list of dictionaries) for each job.

(a) The word "bye" with any capitalization.

(1 pt)

(b) The phrase "what time is it" with any capitalization.

(1 pt)

(c) **Any adjective** followed by the word "cat" (like "fluffy cat").

(1 pt)

3. Rule-based matching

4 points

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)
matcher.add("THANKS", [[{"LOWER": "thank"}, {"LOWER": "you"}], [{"LOWER": "thanks"}]])
doc = nlp("Thanks! I want to thank you, Bot.")
matches = matcher(doc)
for match_id, start, end in matches:
    print(start, end, doc[start:end].text)
```

The tokens are: Thanks (0) ! (1) I (2) want (3) to (4) thank (5) you (6) , (7) Bot (8) . (9)

(a) How many matches are found? (1 pt)

(b) Write the line printed for the **second** match. (2 pts)

(c) Why is the `end` number one **more** than the position of the last matched token? (1 pt)

4. Privacy Shield

4 points

`t.ent_iob_` is **B** for the first token of an entity, **I** for a token inside one, and **O** for a token outside every entity. `t.ent_type_` is the token's entity label ("" if none).

```
doc = nlp("Yesterday Leo Martinez called me.")
out = ""
for t in doc:
    if t.ent_type_ == "PERSON":
        if t.ent_iob_ == "B":
            out = out + "[PERSON] "
    else:
        out = out + t.text_with_ws
print(out)
```

(a) spaCy labels "Yesterday" DATE and "Leo Martinez" PERSON. Fill in `ent_iob_` for each token. (2 pts)

Token	Yesterday	Leo	Martinez	called
<code>ent_iob_</code>				

(b) What does the program print? (2 pts)

5. Teaching spaCy new entities

4 points

```
ruler = nlp.add_pipe("entity_ruler", before="ner")
patterns = [
    {"label": "PET", "pattern": "hamster"},
    {"label": "PET", "pattern": [{"LOWER": "guinea"}, {"LOWER": "pig"}]},
]
ruler.add_patterns(patterns)
doc = nlp("Noah has a hamster and a Guinea Pig in Dallas.")
print([(e.text, e.label_) for e in doc.ents])
```

(a) What does it print? (The model also finds a PERSON and a GPE.) (2 pts)

(b) Would the first pattern match the word "Hamster" with a capital H? Why? (1 pt)

(c) Why add the ruler `before="ner"` instead of after it? (1 pt)

6. Fantasy Map Finder

3 points

On its own, spaCy's model labels **Ember** in "Ember the dragon flew over Glimmerdale." as **PERSON**. A student adds the rule `{"label": "CREATURE", "pattern": "Ember"}` with an entity ruler.

(a) If the ruler is added with `before="ner"`, what label does Ember get? (1 pt)

(b) If the ruler is added with `after="ner"`, what label does Ember get? (1 pt)

(c) The model labels "Glimmerdale" as ORG. Why does it get invented names like this wrong? (1 pt)

7. KidBot Brain

4 points

```
from spacy.matcher import Matcher
matcher = Matcher(nlp.vocab)
matcher.add("BYE", [[{"LOWER": "bye"}], [{"LOWER": "goodbye"}]])
matcher.add("WEATHER", [[{"LOWER": "weather"}]])

def reply(text):
    doc = nlp(text)
    matches = matcher(doc)
    if matches:
        intent = nlp.vocab.strings[matches[0][0]]
        if intent == "BYE":
            return "See you later!"
        if intent == "WEATHER":
            dates = [e.text for e in doc.ents if e.label_ == "DATE"]
            if dates:
                return f"On {dates[0]} it will rain."
            return "Today it is sunny."
    return "Sorry, I don't understand yet."
```

Write what `reply(...)` returns for each message. (spaCy finds "Friday" as a DATE.)

Message	KidBot replies
"How is the weather?"	
"Goodbye!"	
"Sing a song"	
"What's the weather on Friday?"	

8. Thanks intent

5 points

Write code that creates a `Matcher` with a rule named `"THANKS"` that matches either the word `"thanks"` or the phrase `"thank you"` (any capitalization). Then write a function `is_thanks(text)` that **returns** `True` if the text matches and `False` if it doesn't.

END OF ASSESSMENT